

PT04 | 01 - Aprenda os conceitos

State, Props, useState, callbacks, re-renderização e dados derivados

Para que serve esta prática?

A Prática 04 ensina a controlar dados que mudam na tela. O exemplo é um simulador de votação: cada clique altera um contador e o painel mostra o resultado atualizado.

Objetivo 1 - State e Props: entender a diferença entre um valor guardado pelo componente e uma informação recebida por ele.

Objetivo 2 - useState: usar o Hook para guardar contadores e o nome digitado.

Objetivo 3 - atualização segura: usar uma função que recebe o valor anterior antes de somar.

Objetivo 4 - tela atualizada: entender a re-renderização e calcular dados derivados sem duplicar estados.

1. State: a memória que muda

State é um valor que o componente guarda e pode atualizar. No projeto, votosA, votosB, votosC e nomeMesario são estados.

Analogia: imagine três placares e uma prancheta do mesário. Cada placar tem seu próprio número; a prancheta tem seu próprio texto. Alterar um não apaga os outros.

```
const [votosA, setVotosA] = useState(0);  
const [nomeMesario, setNomeMesario] = useState('');
```

useState(0) começa o contador em zero. O primeiro nome é o valor atual; o segundo é a função que pode trocar esse valor.

2. Props: o recado que chega ao componente

Props são informações passadas de um componente pai para um componente filho. A urna entrega nome, votos, total e ação de votar para cada cartão de candidato.

Analogia: a urna é a professora e o cartão Candidato é o aluno. A professora entrega uma ficha com os dados; o aluno usa a ficha para mostrar a informação.

```
function Candidato({ nome, votos, total, onVotar }) {  
  // nome, votos, total e onVotar são props  
}
```

A diferença principal é: state pertence ao componente e pode mudar dentro dele; props chegam de fora e devem ser tratadas como informações recebidas.

3. Atualização segura com callback

Para somar votos, o React permite passar uma função para o setter. Essa função recebe o valor anterior mais recente.

```
setVotosA(prev => prev + 1);
```

Analogia: uma fila. Cada pessoa olha para a última posição e ocupa a próxima. Assim, dois cliques rápidos não tentam usar a mesma contagem antiga.

Evite depender diretamente de `votosA` quando o novo valor depende do valor anterior: **`setVotosA(votosA + 1)`** pode ficar desatualizado quando várias atualizações são agrupadas.

4. Re-renderização: o placar se redesenha

Quando um setter é chamado, o React executa o componente novamente e atualiza o que mudou na tela. O código não precisa localizar cada texto e alterá-lo à mão.

```
clique no botão  
-> setVotosA(prev => prev + 1)  
-> React re-renderiza  
-> o placar mostra o novo voto
```

Analogia: um placar eletrônico. Você registra o ponto; o placar calcula e mostra o número novo.

5. Dados derivados

Um dado derivado é calculado a partir de outros valores. O total e a porcentagem são derivados dos votos atuais, portanto não precisam de `useState` próprio.

```
const totalVotos = votosA + votosB + votosC;  
const percentual = total === 0 ? 0 : (votos / total) * 100;
```

Analogia: uma calculadora. Você guarda os três placares e calcula a soma quando precisa. Guardar também a soma criaria duas folhas que poderiam ficar diferentes.

O teste **`total === 0`** protege o início da tela. Dividir por zero produziria NaN ou Infinity; aqui o resultado inicial é 0%.

6. TextInput controlado

O campo do mesário também usa state. `value` mostra o valor guardado e `onChangeText` atualiza o valor a cada tecla.

```
<TextInput  
  value={nomeMesario}  
  onChangeText={setNomeMesario}  
>
```

Analogia: um espelho. O campo mostra o que está no state, e cada tecla avisa ao state o que mudou.

7. Erros comuns e como pensar

Usar um state gigante para tudo

Pode funcionar, mas mistura informações independentes. Nesta aula, cada contador e o nome do mesário têm seu próprio state, deixando as mudanças mais claras.

Criar state para o total

Não é necessário. O total sempre pode ser obtido pela soma dos três contadores. Menos cópias significa menos chance de inconsistência.

Esquecer o caso de total zero

Sempre proteja cálculos que dividem pelo total. O painel deve começar mostrando 0%, nunca NaN.

Alterar props diretamente

O componente Candidato recebe dados e mostra a ficha. Quem atualiza os votos é a urna, por meio da função onVotar.

Mapa mental da urna

```
state guarda -> botão chama -> callback soma
  -> React re-renderiza -> total e porcentagens são recalculados
  -> a tela mostra o resultado
```

Se você consegue explicar esse caminho com suas palavras, entendeu o núcleo da prática.

Resumo de uma frase

State é a memória da tela, props são recados recebidos, callback atualiza usando o valor mais recente e dados derivados são contas feitas a partir da memória existente.